

Documentación Técnica de Backend, Procesos y Reglas de Negocio (Controladores)

Este documento detalla la arquitectura de backend, los controladores de Express, sus endpoints, modelos de datos de Mongoose, servicios internos y las reglas y flujos de negocio que orquestan en el sistema **API Costeo**.

Mapa General de Enrutamiento (Express)

El ruteador principal (`src/api/routes/index.route.ts`) centraliza y redirige el tráfico HTTP hacia los diferentes controladores mediante prefijos de ruta bajo `/api-costeo` :

Prefijo de Ruta	Controlador de Express	Clase de TS	Archivos de Código
<code>/ia-analyzer</code>	<code>ia.controller.ts</code>	<code>IAController</code>	ia.controller.ts
<code>/traceability</code>	<code>TraceabilityController.controller.ts</code>	<code>TraceabilityController</code>	TraceabilityController.controller.ts
<code>/daily-reports/scan</code>	<code>DailyReportScannerController.controller.ts</code>	<code>DailyReportScannerController</code>	DailyReportScannerController.controller.ts
<code>/reports</code>	<code>report.controller.ts</code>	<code>ReportController</code>	report.controller.ts
<code>/departures</code>	<code>departure.controller.ts</code>	<code>DepartureController</code>	departure.controller.ts
<code>/baseTeb</code>	<code>BaseTebController.controller.ts</code>	<code>BaseTebController</code>	BaseTebController.controller.ts
<code>/fictitiousteb</code>	<code>fictitiousTeb.controller.ts</code>	<code>FictitiousTebController</code>	fictitiousTeb.controller.ts
<code>/dashboard</code>	<code>dashboard.controller.ts</code> y <code>dashboardMetric.controller.ts</code>	<code>dashboardController</code> / <code>DashboardMetricController</code>	dashboard.controller.ts
<code>/configs</code>	<code>Config.controller.ts</code>	<code>ConfigController</code>	Config.controller.ts

/generator	GeneratorController.controller.ts	GeneratorController	GeneratorController.controller.ts
/jorndays	jornDay.controller.ts	JornDayController	jornDay.controller.ts
/resources	resource.controller.ts	ResourceController	resource.controller.ts
/catalogequipments	CatalogEquipment.controller.ts	CatalogEquipmentController	CatalogEquipment.controller.ts
/activities	activity.controller.ts	ActivityController	activity.controller.ts
/connections	connection.controller.ts	ConnectionController	connection.controller.ts
/anuncios	AnuncioController.controller.ts	AnuncioController	AnuncioController.controller.ts

1. Módulo IA-Analista (`iaAnalistaController`)

A. Lógica y Procesos de Negocio

- **Auditoría Cruzada de Reportes:** Automatiza el análisis de KPIs y finanzas del contrato. Claude (Anthropic) genera análisis y redacciones con estilo ejecutivo y estratégico ("Estilo Nicolás"). Gemini (Google Vertex AI) realiza la auditoría numérica validando los números contra la base de datos de origen, corrigiendo discrepancias de forma transparente en el texto final.
- **Modo Macro y Micro:** Segmenta reportes en globales (Macro, de 8 puntos) o específicos a un KPI del dashboard (Micro, enfocado en mantenimiento o fletes).
- **Asincronía Operativa:** Dado que la redacción y auditoría cruzada con LLMs puede demorar entre 15 y 45 segundos, se utiliza un sistema de trabajos en segundo plano (Jobs).

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `IAController`
- **Servicios Relacionados:** `IAJobService`, `ClaudeService`, `ValidatorService`, `TransformerService`.
- **Modelos de Mongoose:** `IAJobModel` (guarda estado `pending`, `processing`, `completed`, `failed`), `DashboardMetricModel` (recupera `AI_GLOBAL_CONFIG`).
- **Flujo y Métodos de la Clase:**
 - `analyze` (POST `/ia-analyzer/analyze`): Crea un UUID de Job en `IAJobModel`, inicia `processAnalyzeJob` en segundo plano sin esperar a la IA y retorna un HTTP **202 Accepted** al cliente con el `jobId`.

- `analysisLive` (POST `/ia-analyzer/analysisLive`): Similar a `analyze`, pero inicia el job `processAnalysisLiveJob` para consolidar datos contables calculados al vuelo en un rango de fechas.
 - `checkStatus` (GET `/ia-analyzer/status/:jobId`): Método de consulta periódica (polling). Devuelve un HTTP **202** si el trabajo está pendiente, un HTTP **200** con el resultado consolidado si está completado, o un HTTP **500** si falló la IA.
-

2. Consolidación de Costos (TraceabilityController)

A. Lógica y Procesos de Negocio

- **Sandbox de Previsualización:** Evita colisionar o corromper datos de cobros contables reales. El controlador permite crear una sesión temporal para que los analistas simulen, verifiquen y editen partidas, volúmenes o asignaciones de costos manualmente.
- **Regla de Consolidación (Commit):** Al confirmar la sesión, los datos se clasifican y guardan:
 - *Trazabilidad Real:* Si el servicio tiene un número de solicitud presupuestal oficial (TEB/CAB/CAX), se guarda en `TrazabilityHistoryModel`.
 - *Prorrrateo:* Si el campo de solicitud es nulo o temporal, se guarda en `TraceabilityProrrrateoModel` para redistribución de costos.
- **Clave Compuesta:** Evita duplicaciones cruzando `día | equipo | partida | exterior/interior | ID de actividad`.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `TraceabilityController`
 - **Servicios:** `TrazabilityService`.
 - **Modelos:** `TrazabilityHistoryModel`, `TraceabilityProrrrateoModel`, `TrazabilityTempModel`, `ExportTaskModel`.
 - **Flujo y Métodos de la Clase:**
 - `generateAndStoreTemp` (POST `/traceability/generate-temp`): Llama a `generateAndStoreInTemp` en el rango de fechas. Registra los ítems calculados en `TrazabilityTempModel` con estatus `PENDING`.
 - `listTempTraceability` (GET `/traceability/temp/:sessionId`): Recupera registros temporales paginados aplicando filtros específicos de equipos o partidas.
 - `commitTempTraceability` (POST `/traceability/commit/:sessionId`): Traslada de forma segura los registros temporales a los históricos utilizando escrituras en bloque controladas.
 - `startAsyncExport` (POST `/traceability/export-async`): Registra una tarea en `ExportTaskModel` e inicia `TrazabilityService.generateExcelAsync` en segundo plano para exportar grandes historiales a Excel sin bloquear el hilo del servidor.
-

3. Escaneo y OCR de Bitácoras (DailyReportController)

A. Lógica y Procesos de Negocio

- **Digitalización Inteligente:** Convierte partes diarios escaneados en partes estructurados digitales. Segmenta PDFs de múltiples páginas, subiendo folios independientes a Cloudinary.
- **Notificación Concurrente:** Emite eventos progresivos a través de websockets por oficina (`officeId`) para que el usuario visualice el estado de digitalización de su lote de reportes.
- **Límite de Cabos:** Restringe por negocio que un reporte tipo cabos (`caboServiceDay: true`) contenga como máximo 1 ID en `caboIds`.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `DailyReportScannerController`
 - **Servicios:** `DailyReportGeminiService`, `tempDailyReportsService`.
 - **Modelos:** `DailyReportScannerSessionModel` (guarda estado de procesamiento paralelos de PDFs), `TempDailyReportModel` (reportes provisionales).
 - **Flujo y Métodos de la Clase:**
 - `uploadAndExtract` (POST `/daily-reports/scan/upload`): Recibe archivos desde el frontend (`multer`). Crea una sesión en `DailyReportScannerSessionModel`, divide archivos grandes (`splitFileIfNeededReports`) y lanza en segundo plano `processFilesBackground` retornando un HTTP **202**.
 - `processFilesBackground` (Privado): Controla la concurrencia (`processInParallel`) al enviar URLs seguras de Cloudinary al servicio de Gemini para estructurar los datos del reporte. Notifica el progreso vía sockets.
 - `receiveFromScanner` (POST `/daily-reports/scan/scanner`): Endpoint para que los dispositivos físicos de escáner envíen bitácoras directamente en formato binario.
-

4. Gestión de Partes Diarios (ReportControllers)

A. Lógica y Procesos de Negocio

- **Regla de Límite Diario de Jornada:** Valida de forma obligatoria que la sumatoria de fracciones operativas de un activo en una fecha (`Operación + Disponibilidad + Mantenimiento + Fuera de Servicio`) sea exactamente **1.00**.
- **Recálculo en Cascada:** Cualquier cambio de reporte (alta/baja/modificación) dispara un recálculo sobre los `laborDays` de `SubItemsModel` asociados al activo para actualizar de forma inmediata las estimaciones financieras del contrato.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `ReportController`
 - **Servicios:** `reportService`, `ReportSyncService`, `DepartureService`.
 - **Modelos:** `ReportModel`, `ActivityModel`, `SubItemsModel`.
 - **Flujo y Métodos de la Clase:**
 - `createReport` (POST `/reports`): Registra la cabecera del parte diario, asocia actividades, verifica el límite de 1.0 jornada diaria y recalcula en base de datos.
 - `findAll` (GET `/reports`): Paginación y búsqueda server-side. Parsea filtros directos (`filters[campo]`) y filtros avanzados dinámicos (`customFilters[index][column/operator/value]`) decodificando caracteres y formateando la query en MongoDB.
 - `validateReportVolume` (POST `/reports/validate-volume`): Llama a `reportService.validateReportVolumeBeforeCreate` para responder si el activo tiene volumen disponible o choca con otro reporte en la fecha.
-

5. Partidas Contractuales (DepartureControllers)

A. Lógica y Procesos de Negocio

- **Reglas de Tarifa por Concepto:**
 - *Fuera de Instalación (FS):* Aplica de forma automática un **descuento del 75%** sobre la tarifa unitaria cuando el equipo realiza actividades fuera de sitio (`isOutside = true`).
 - *Mantenimiento:* Registra las jornadas en taller a costo de cero.

- **Kilometraje en Fletes:** Evalúa independientemente si la distancia del servicio cae en el rango de la partida para asignar volumen 1, transfiriendo los kilómetros excedentes para el cálculo del siguiente bloque.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `DepartureController`
 - **Servicios:** `DepartureService`.
 - **Modelos:** `DepartureModel`, `SubItemsModel`.
 - **Flujo y Métodos de la Clase:**
 - `addSubItemInDeparture` (POST `/departures/subitem`): Vincula un equipo (`CatalogEquipment`) a la partida contractual y reserva las estructuras de jornadas (`laborDays`).
 - `getHierarchicalData` (GET `/departures/hierarchical/:month/:year`): Construye la vista de árbol contractual (Partida -> Equipos -> Días trabajados y montos acumulados por mes) utilizada directamente en el frontend.
-

6. Documentos Presupuestales (`BaseTebController`)

A. Lógica y Procesos de Negocio

- **OCR en Paralelo Balanceado:** Extrae los campos de PDFs de TEB utilizando Azure Form Recognizer a través de un pool balanceado con estrategia round-robin.
- **White Paper ZIP:** Consolida e integra los presupuestos autorizados agrupando TEBs por programa presupuestal (PEP). Combina los documentos PDF físicos de respaldo en archivos ZIP estructurados.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `BaseTebController`
 - **Servicios:** `BaseTebService`, `AzureOcrPoolService`.
 - **Modelos:** `BaseTebModel`, `FictitiousTebModel`, `FictitiousTebPivotModel`.
 - **Flujo y Métodos de la Clase:**
 - `ReceiveFilesAndUploadToAzure` (POST `/baseTeb/upload`): Sube los archivos a Cloudinary, segmenta páginas y realiza llamadas al pool de Azure para extraer y formatear campos.
 - `importExcelChunk` (POST `/baseTeb/import-excel-chunk`): Recibe lotes de Excel de hasta 500 registros y los inserta de forma idempotente con `bulkWrite`.
 - `exportTebWhitePaperZip` (POST `/baseTeb/export-whitepaper-zip`): Utiliza `archiver` para comprimir en ZIP los archivos PDF cargados de Cloudinary combinados con `pdf-lib` por PEP.
-

7. TEBs Provisionales (`FictitiousTebController`)

A. Lógica y Procesos de Negocio

- **Continuidad Operativa:** Permite crear registros presupuestales provisionales ("TEBs Ficticios") con folios provisionales autogenerados para evitar retrasos en la captura de bitácoras de trabajo diarios si el cliente aún no ha entregado las órdenes de servicio oficiales.
- **Migración Automática:** Al cargarse el TEB definitivo, asocia los partes de trabajo capturados con el presupuesto final.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `FictitiousTebController`
- **Servicios:** `fictitiousTebService`.
- **Modelos:** `FictitiousTebModel`.

- **Flujo y Métodos de la Clase:**

- `createFictitiousTeb` (POST `/fictitiousteb`): Guarda el registro simulado asociándole las imágenes de Cloudinary cargadas por el capturista.
- `getLastFolio` (GET `/fictitiousteb/last-folio`): Consulta el consecutivo en base de datos para autogenerar folios sin solapar registros.

8. Dashboard y Estadísticas (`dashboardController`)

A. Lógica y Procesos de Negocio

- **Agregaciones en Tiempo Real:** Agrupa datos de costos unitarios y volumen real de trazabilidad contable para alimentar los gráficos visuales de dispersión, desviaciones del presupuesto y costos de inactividad por mantenimiento o exterior.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `dashboardController`
- **Servicios:** `dashboardService`.
- **Modelos:** `TrazabilityHistoryModel`, `dashboardModel`.
- **Flujo y Métodos de la Clase:**
 - `loadDashboardScatter` (POST `/dashboard/scatter`): Realiza una agregación avanzada para mapear el costo y servicios de los activos.
 - `getTotalCostsComparedWithGoal` (GET `/dashboard/costs-goal`): Compara el costo acumulado contra el tope del presupuesto objetivo.

9. Métricas y Programación de Alertas (`DashboardMetricController`)

A. Lógica y Procesos de Negocio

- **Alertas Dinámicas:** Administra los umbrales de alarmas de desviación y recarga de forma transparente las tareas programadas (Cron Jobs) al actualizar la configuración.
- **Reportes de Alerta:** Automatiza el envío de correos directos con reportes de desvíos financieros por partida al correo de los administradores.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `DashboardMetricController`
- **Servicios:** `dashboardMetricService`.
- **Modelos:** `DashboardMetricModel`.
- **Flujo y Métodos de la Clase:**
 - `upsertMetric` (POST `/dashboard/metrics`): Inserta/actualiza umbrales y llama a `initCronScheduler` para reconstruir las tareas programadas de Node-Cron en memoria sin parar el servidor.
 - `testCronJob` (POST `/dashboard/metrics/cron/test/:metricId`): Ejecuta el cron job en background (`executeCronJob`) para enviar un correo de prueba de manera inmediata.

10. Catálogos Dinámicos (`ConfigController`)

A. Lógica y Procesos de Negocio

- **Estandarización de Variables:** Administra catálogos de instalaciones de destino, de servicio y nombres de barcos autorizados. Las cargas masivas de catálogos se realizan mediante operaciones masivas idempotentes para evitar el impacto de rendimiento del patrón de guardado uno a uno.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `ConfigController`
 - **Servicios:** `ConfigService`.
 - **Modelos:** `CatalogListModel`, `ConfigurationModel`.
 - **Flujo y Métodos de la Clase:**
 - `bulkCreateCatalogList` (POST `/configs/catalog/bulk`): Recibe un arreglo de datos e inyecta masivamente registros utilizando `bulkWrite` y `updateOne + upsert`.
 - `getListByCodeFilter` (GET `/configs/catalog/filter/:code`): Recupera e implementa paginación server-side con búsquedas de catálogo.
-

11. Generador de Formularios y Plantillas (`GeneratorController`)

A. Lógica y Procesos de Negocio

- **Decoración de Formularios:** Controla la visualización dinámica del formulario de partes diarios en función del equipo. Determina qué campos de volumen de horas y kilómetros habilitar en base a las partidas contractuales asociadas al activo.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `GeneratorController`
 - **Servicios:** `CatalogEquipmentService`, `DepartureService`, `reportService`.
 - **Modelos:** `CatalogEquipmentModel`, `DepartureModel`.
 - **Flujo y Métodos de la Clase:**
 - `determinateShowInputs` (GET `/generator/inputs/:equipmentId`): Resuelve las partidas asociadas al activo y retorna qué inputs de horas y kilometraje son válidos.
 - `generateExcel` (POST `/generator/excel`): Llama a `transformTebData` y responde enviando un archivo binario de Excel de plantilla de TEB.
-

12. Turnos de Trabajo (`JornDayControllers`)

A. Lógica y Procesos de Negocio

- **Horarios Contractuales:** Permite configurar e integrar turnos operativos permitidos en contrato (matutino, diurno, vespertino, nocturno, mixto). Los límites del turno validan que las horas reportadas en las actividades no se solapen.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `JornDayController`
 - **Servicios:** `jornDayService`.
 - **Modelos:** `JornDayModel`.
 - **Flujo y Métodos de la Clase:**
 - `createJornDay` (POST `/jorndays`): Valida que los campos `description`, `duration`, `dayEsp`, `startTime` y `endTime` estén completos antes de registrar la jornada.
-

13. Recursos y Capacidades (ResourceControllers)

A. Lógica y Procesos de Negocio

- **Matching Contractual de Capacidad:** Registra capacidades físicas (tonelajes, longitudes de plumas) asociadas a tipos de recursos, permitiendo verificar si la grúa o equipo cumple con el perfil contractual exigido en la partida.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `ResourceController`
 - **Servicios:** `resourceService`.
 - **Modelos:** `ResourceModel`.
 - **Flujo y Métodos de la Clase:**
 - `createResource` (POST `/resources`): Guarda las especificaciones técnicas (`catalog`, `capacity`, `unit`, `specification`).
-

14. Catálogo de Activos (CatalogEquipmentController)

A. Lógica y Procesos de Negocio

- **Asignación de Cuadrillas:** Vincula de manera jerárquica un Cabo líder con su cuadrilla de Maniobristas.
- **Validaciones Físicas de Activos:** Valida si el equipo está autorizado para ser embarcado en el barco correspondiente.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `CatalogEquipmentController`
 - **Servicios:** `CatalogEquipmentService`.
 - **Modelos:** `CatalogEquipmentModel`, `CatalogConceptModel`.
 - **Flujo y Métodos de la Clase:**
 - `assignCaboAndManeuverer` (POST `/catalogequipments/assign-cabo-maniobrista`): Vincula un Cabo con un listado de IDs de Maniobristas en base de datos.
 - `validateEquipmentToShip` (GET `/catalogequipments/validate-ship/:equipmentId/:blockDepartures`): Comprueba si el equipo cumple con las condiciones para ser embarcado en el buque especificado.
-

15. Actividades de Trabajo (ActivityController)

A. Lógica y Procesos de Negocio

- **Desglose Quirúrgico de Horas:** Controla el registro detallado de las tareas del activo durante su turno diario, asegurando que se guarden los horarios de inicio y fin, el tipo (OP, DISP, MTTO, FS) y sus respectivos códigos presupuestales (TEB/CAB/CAX).

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `ActivityController`
 - **Servicios:** `activityService`.
 - **Modelos:** `ActivityModel`.
 - **Flujo y Métodos de la Clase:**
 - `createActivity` (POST `/activities`): Valida y registra la actividad diaria asociada a la bitácora de cabecera.
-

16. Conexión de Servicios (`ConnectionController`)

A. Lógica y Procesos de Negocio

- **Centralización de Oficinas:** Administra y asocia los perfiles de conexión de las bases de datos transaccionales correspondientes a las diferentes sucursales/oficinas de la empresa.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `ConnectionController`
 - **Servicios:** `connectionService`.
 - **Modelos:** `ConnectionProfileModel`.
 - **Flujo y Métodos de la Clase:**
 - `createConnection` (POST `/connections`): Registra y guarda un perfil de conexión parametrizado (`officeId`, `selectedProfile`, `profiles`) en la base de datos de configuración global.
-

17. Anuncios y Comunicados (`AnuncioController`)

A. Lógica y Procesos de Negocio

- **Comunicación Interna:** Gestiona anuncios, noticias y alertas sobre cierres de facturación o mantenimientos de la plataforma en la pantalla principal.

B. Especificación del Backend y Flujo Técnico

- **Controlador:** `AnuncioController`
- **Modelos:** `AnuncioModel`.
- **Flujo y Métodos de la Clase:**
 - `getAnuncios` (GET `/anuncios`): Recupera las notificaciones ordenadas por fecha que se encuentran marcadas con el flag de activas (`active: true`).